# ZYNXIS CLIENT PORTAL

## Technical Report

*Full-Stack Development Internship — Capstone Project*

| Field | Detail |
|---|---|
| Project | Zynxis Client Portal |
| Type | Full-stack web application |
| Frontend | React + Vite + Tailwind CSS |
| Backend | Node.js + Express.js |
| Database | MongoDB + Mongoose |
| Authentication | JWT-based authentication and protected routes |
| Deployment | Vercel (frontend and backend/API) |
| API Documentation | Postman-compatible collection included in repository |

---

## 1. Executive Summary

The Zynxis Client Portal is a full-stack web application designed to give clients a centralized interface for viewing projects, monitoring tasks, receiving notifications, and interacting with project-related attachments. The application was developed as the capstone project of a full-stack development internship and demonstrates the integration of a React frontend, an Express/Node.js REST API, MongoDB persistence, JWT authentication, and production deployment.

The system follows a client-server architecture. The React frontend communicates with the backend through REST endpoints. The backend validates requests, authenticates users, enforces authorization, performs database operations, generates notifications, and processes file uploads. MongoDB stores the application's core entities and their relationships.

## 2. Project Objectives

- Build a practical client-facing portal using a modern JavaScript full-stack.
- Implement secure login, logout, JWT authentication, and protected routes.
- Provide project and task information through reusable frontend components.
- Implement server-driven notifications and unread notification counts.
- Support task attachment uploads and attachment viewing.
- Connect frontend data fetching with backend APIs and MongoDB.
- Deploy the completed application and document the API.

## 3. High-Level Architecture

| Layer | Technology / Responsibility |
|---|---|
| Presentation | React, React Router and Tailwind CSS; Login, Dashboard, Projects, Tasks and Notifications pages. |
| Data Fetching | Axios and TanStack React Query for requests, caching, loading/error states and query invalidation. |
| API | Node.js + Express REST endpoints for authentication, projects, tasks, dashboard and notifications. |
| Security | JWT authentication middleware and authorization checks for protected operations. |
| Persistence | MongoDB accessed through Mongoose models and database queries. |
| File Handling | Multer multipart processing with generated filenames and a 5 MB limit. |

## 4. Backend Architecture

The backend is implemented with Node.js and Express.js. The server loads environment configuration, connects to MongoDB, enables CORS and JSON parsing, and mounts feature-specific route modules. Authentication, projects, tasks, dashboard statistics and notifications are separated into route/controller responsibilities to keep the API organized and maintainable.

Core backend responsibilities include:

- User registration and login.
- JWT verification for protected resources.
- Authorization checks for restricted operations.
- Project and task data retrieval and creation.
- Dashboard statistics and notification retrieval.
- Multipart attachment upload processing through Multer.

## 5. Database Design

MongoDB is used as the primary database, with Mongoose providing schemas and model-level interaction. The main business entities are separated into collections/documents so that projects can reference clients, tasks can reference projects and users, and users can be assigned to both projects and tasks.

| Entity | Purpose | Important Data |
|---|---|---|
| User | Stores portal users and authentication information. | fullName, email, password, role |
| Client | Represents client records. | client information and related data |
| Project | Stores client projects and their state. | projectName, status, client, assignedUsers, budget, deadline |
| Task | Stores project work items. | title, description, project, assignedTo, priority, status, dueDate, attachment |
| Notification | Stores system events shown to users. | message, type, isRead, createdAt |

## 6. REST API Design

The API is organized by resource. Protected endpoints receive a JWT through the Authorization header using the Bearer scheme.

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /api/auth/login | Authenticate a user. |
| POST | /api/auth/register | Register a new user. Restricted to admin users. |
| GET | /api/projects | Retrieve project data. |
| GET | /api/dashboard | Retrieve dashboard statistics. |
| GET / POST | /api/tasks | Retrieve and create tasks. |
| POST | /api/tasks/:id/upload | Upload a task attachment. |
| GET | /api/notifications | Retrieve notifications. |

## 7. Authentication and Authorization

Authentication is based on JSON Web Tokens. After successful login, the frontend stores the returned token and sends it with protected requests. Backend middleware verifies the token before allowing access to protected resources. Authorization checks are applied where operations require appropriate access — for example, the user registration endpoint is restricted to authenticated admin users. Invalid or expired tokens are rejected.

Environment secrets are kept outside source control through environment configuration and .gitignore rules. This separates development/deployment configuration from application source code.

## 8. Frontend Architecture

The frontend is built with React and Vite. Tailwind CSS provides styling and React Router manages navigation between authenticated pages. Reusable components such as DataTable and FormInput reduce duplication and help maintain consistent UI behavior.

Main frontend pages:

| Page | Functionality |
|---|---|
| Login | Authenticates the user and establishes the session token. |
| Dashboard | Displays total clients, projects, tasks and unread notifications. |
| Projects | Displays project cards and a project overview table. |
| Tasks | Displays task details, status, due date and attachments. |
| Notifications | Displays messages, timestamps and unread state. |
| Main Layout | Provides navigation and logout functionality. |

## 9. State and Data Fetching

TanStack React Query is used for server-state management. Dashboard, project, task and notification data are fetched from the API through query functions. Loading and error states are handled explicitly. Mutations are used for task attachment uploads, and successful mutations trigger query invalidation so the UI reflects backend changes.

## 10. File Upload System

The task module supports attachments through Multer. Files are submitted as multipart/form-data under the field name attachment. The backend stores files in the configured uploads directory and generates unique filenames using a timestamp, random value and original extension. A 5 MB file-size limit is configured.

The frontend provides a file selector and Upload Attachment button for each task. After a successful upload, the task shows an uploaded indicator and a View Attachment link. The upload mutation refreshes task data after completion.

## 11. Notification System

The portal implements server-driven notifications for important events. Task creation produces a notification that is available through the notifications endpoint. The interface distinguishes unread notifications and the dashboard calculates the unread count.

## 12. Dashboard and Reporting Features

The dashboard retrieves aggregate counts from the backend and displays total clients, total projects, total tasks and unread notifications. The dashboard controller performs database count operations in parallel to efficiently obtain these statistics.

## 13. Responsive User Interface

The interface uses a consistent navigation sidebar, cards, tables, spacing and typography. Tailwind responsive utilities allow project and task layouts to adapt to different screen sizes. The design focuses on clear hierarchy and direct access to portal functions.

## 14. Testing and Verification

The application was tested progressively during development. Frontend pages were verified against live backend responses, while API requests were inspected to confirm authentication behavior, HTTP status codes and returned JSON data.

| Area | Verification | Result |
|---|---|---|
| Authentication | Login, logout, protected access and invalid/expired token behavior. | Verified |
| Projects | Project data retrieved from API and rendered in cards/table. | Verified |
| Tasks | Task retrieval, creation, project association and overview. | Verified |
| File Upload | Multipart upload, stored filename, uploaded state and link. | Verified |

| Area | Verification | Result |
|---|---|---|
| Notifications | Notification retrieval and unread dashboard count. | Verified |
| Dashboard | Live aggregate counts returned by backend. | Verified |
| API Collection | Endpoints documented in Postman-compatible JSON. | Included |

## 15. Deployment

The completed portal was prepared for production deployment with environment-based configuration. Both the frontend and the backend/API are hosted on Vercel; the deployed frontend communicates with the deployed backend through the configured VITE_API_URL for all production requests. Local development values remain separate from deployment values.

**Production URL**

**Backend/API:** https://zynxis-client-portal.vercel.app

**Frontend/API:** https://zynxis-client-portal-98lr.vercel.app

## 16. Security Considerations

- JWT tokens are required for protected API resources.
- Authorization middleware prevents unauthorized access to protected operations, including restricting user registration to admin users.
- Passwords and environment secrets are excluded from Git.
- Uploaded files are subject to a 5 MB size limit.
- Invalid or expired authentication tokens result in rejected requests.

## 17. Project Structure

| Directory | Role |
|---|---|
| backend/ | Express API, controllers, routes, middleware, models and server configuration. |
| frontend/ | React application, pages, layouts and reusable components. |
| postman/ | Postman-compatible API collection JSON. |
| docs/ | Project documentation and supporting deliverables. |

## 18. Conclusion

The Zynxis Client Portal demonstrates the complete development of a modern full-stack web application from architecture and database integration through REST API development, authentication, responsive frontend implementation, live data fetching, notifications, file uploads and deployment. The project also includes API documentation in a Postman-compatible collection and a structured repository suitable for continued development.

The system provides a practical foundation for a client portal and can be extended with richer role-based dashboards, project creation workflows, task status updates, search/filtering, audit logs, cloud object storage and automated testing.

---

**Prepared for:** Full-Stack Development Internship — Capstone Submission
**Project:** Zynxis Client Portal
**Author:** Ayeza Waheed